

SERVICIO NACIONAL DE APRENDIZAJE SENA

Centro de Gestión de Mercados, Logística y TIC



APP TRUEQUE MATCH

**Evidencia GA8-220501096-AA2-EV01. Taller. Integración,
tecnologías emergentes y disruptivas.**

Estudiante: Gerson Jonnathan Lopez Oviedo

Ficha: 3186647

Instructor: Arnaldo Alfonso Montiel Brun

Programa: Tecnólogo en Análisis y Desarrollo de Software

Bogotá. Colombia 2026 julio 2026

Introducción

Este documento presenta la evidencia GA8-220501096-AA2-EV01, un taller dividido en dos secciones. En la primera investigo y explico, con mis propias palabras y ejemplos sencillos, los conceptos fundamentales del desarrollo de aplicaciones móviles para Android. En la segunda, aplico esos conceptos de forma práctica creando mi primera aplicación Android desde cero directamente en Android Studio, sin pasar por Expo ni React Native, para demostrar el dominio de la herramienta nativa de desarrollo antes de seguir trabajando sobre el proyecto Trueque Match.

Sección 1. Investigación sobre Desarrollo de Aplicaciones Móviles para Android

¿Qué es Android?

Android es un sistema operativo diseñado principalmente para dispositivos móviles con pantallas táctiles, como teléfonos inteligentes y tabletas. Su función principal es administrar los recursos de hardware del dispositivo (procesador, memoria, cámaras) y permitir que las aplicaciones instaladas funcionen e interactúen con el usuario de manera eficiente.

Ejemplo: imaginemos que un teléfono celular es una casa vacía con bombillos, tuberías y electrodomésticos. Android es el administrador o el "mayordomo" de esa casa. Si tú quieres encender la luz (la pantalla) o abrir una llave (la cámara), no tienes que manipular los cables directamente; le pides al mayordomo que lo haga por ti. Él se encarga de que todo funcione de manera coordinada.

Definición del Concepto de APK

APK es la abreviatura de Android Application Package (Paquete de Aplicación Android). Es el formato de archivo que se utiliza para distribuir e instalar aplicaciones en el sistema operativo Android. Contiene todos los elementos necesarios para que la aplicación funcione en el celular, incluyendo el código compilado, las imágenes, los sonidos y los archivos de configuración.

Ejemplo: un archivo APK es exactamente igual a una caja de un mueble para armar que compras en el almacén. Dentro de esa caja viene la madera, los tornillos, las herramientas y el manual de instrucciones. Hasta que no abres la caja e instalas el mueble (le das clic al APK en tu celular), no puedes usar la silla o la mesa. Es el paquete cerrado listo para instalarse.

¿Qué es el Android SDK?

El Android SDK (Software Development Kit o Kit de Desarrollo de Software) es un conjunto de herramientas, librerías, códigos prefabricados y programas de prueba que Google les da a los desarrolladores para que puedan crear aplicaciones específicamente para su sistema operativo.

Ejemplo: imagina que quieres construir una mesa de madera desde cero. Podrías cortar el árbol tú mismo, pero es más fácil si vas a una ferretería y compras un "kit de carpintería" que ya trae el martillo, los clavos a la medida, el pegamento especial y las plantillas con las formas de las patas. El SDK es ese kit de herramientas que evita que el programador tenga que inventar la rueda cada vez que escribe código.

¿Cuál es el Lenguaje Utilizado para Desarrollar Aplicaciones para Android?

Históricamente, el lenguaje rey para programar en Android fue Java. Sin embargo, hoy en día el lenguaje oficial y más recomendado por Google es Kotlin. Ambos lenguajes coexisten y se pueden usar para estructurar las aplicaciones, permitiendo que las instrucciones se traduzcan a un formato que el procesador del celular pueda ejecutar.

Ejemplo: pensemos en las aplicaciones como cartas que le vas a escribir al celular para darle órdenes. Java es como el español clásico y formal que todo el mundo entiende en la oficina, mientras que Kotlin es como un español más moderno, corto y directo. Con ambos puedes dar la misma orden, pero con Kotlin escribes menos líneas y terminas más rápido.

¿Qué IDEs de Desarrollo Existen para Codificar?

Un IDE (Integrated Development Environment o Entorno de Desarrollo Integrado) es el programa de computadora donde el programador se sienta a escribir el código. Los más utilizados para Android son:

- Android Studio: es el IDE oficial de Google, diseñado exclusivamente para hacer aplicaciones de Android.
- Visual Studio Code: un editor muy ligero y popular que, mediante el uso de extensiones, permite programar para entornos móviles.

Ejemplo: un IDE es como el cuaderno de un escritor o el taller de un pintor. Podrías escribir una novela en servilletas de papel, pero es mucho mejor si usas un programa como Word que te resalta los errores de ortografía, te numera las páginas y te organiza los capítulos. El IDE te ayuda a escribir código sin equivocarte en las etiquetas o funciones.

Definición del concepto de android multiusuario

El concepto de Android multiusuario es una característica del sistema operativo que permite que varias personas compartan el mismo dispositivo físico (como una tableta familiar), pero manteniendo sus cuentas, aplicaciones, configuraciones, fondos de pantalla y fotos completamente separados y privados los unos de los otros.

Ejemplo: es como un casillero de un gimnasio que tiene varias llaves distintas. Si entra el Usuario A con su llave, solo ve su ropa y sus cuadernos. Si entra el Usuario B, ve cosas totalmente diferentes. Aunque usan el mismo mueble físico (el celular), las vidas digitales de ambos no se mezclan para nada.

Definición del Concepto de Mínimo Privilegio

El principio de mínimo privilegio es una regla de oro en la seguridad informática que dicta que una aplicación o usuario solo debe tener acceso a los recursos y permisos que son estrictamente necesarios para cumplir con su función específica, y a nada más.

Ejemplo: imaginemos que contratas a un pintor para que arregle la fachada de tu casa. Le das la llave de la puerta principal para que entre al jardín, pero no tienes por qué darle la llave de tu caja fuerte o de tu habitación. Si una aplicación de linterna en tu celular te pide permiso para ver tus fotos y conocer tu ubicación, está violando este principio, porque una linterna solo necesita activar el flash del teléfono.

¿Cuáles son los Componentes Esenciales de una Aplicación Android?

Las aplicaciones de Android no son un bloque de código único, sino que se arman como un rompecabezas de cuatro componentes esenciales:

- **Activities (Actividades):** son las pantallas visuales que el usuario puede ver y con las que puede interactuar (ej. la pantalla de login).
- **Services (Servicios):** son procesos que se ejecutan escondidos en el fondo, sin una interfaz visual (ej. la música que sigue sonando cuando bloqueas el celular).
- **Broadcast Receivers (Receptores de difusión):** componentes que escuchan avisos del sistema (ej. un aviso que le dice a la app que la batería se está agotando).
- **Content Providers (Proveedores de contenido):** son los encargados de administrar y compartir datos entre diferentes aplicaciones de forma segura (ej. cuando WhatsApp te pide permiso para leer la lista de contactos de tu teléfono).

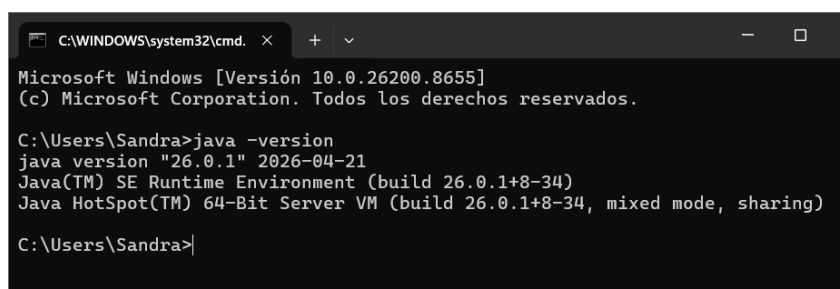
Ejemplo: imagina el funcionamiento de un restaurante. Las Activities son las mesas y la decoración del comedor que los clientes ven. Los Services son los cocineros que trabajan en la parte de atrás ocultos a la vista. El Broadcast Receiver es el timbre que suena cuando entra un cliente nuevo por la puerta. Y el Content Provider es el almacén central de ingredientes de donde todos sacan la comida organizada para preparar los platos.

Sección 2. Creación de mi Primera Aplicación Android

En esta sección documento el proceso completo de creación de una aplicación Android básica directamente en Android Studio, sin pasar por Expo ni React Native, con el fin de demostrar el manejo de la herramienta nativa de desarrollo Android desde cero: instalación de Java, instalación de Android Studio, creación del proyecto, creación de la actividad principal, configuración del proyecto, compilación y ejecución mostrando un mensaje en pantalla.

2.1 Instalación de Java (JDK)

Android Studio necesita el Kit de Desarrollo de Java (JDK) para poder compilar el código, ya que las herramientas de compilación de Android (Gradle) están construidas sobre la máquina virtual de Java. Descargué e instalé el JDK, y confirmé la instalación correcta abriendo una terminal y ejecutando el comando `java -version`.



```
C:\WINDOWS\system32\cmd. x + v
Microsoft Windows [Versión 10.0.26200.8655]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Sandra>java -version
java version "26.0.1" 2026-04-21
Java(TM) SE Runtime Environment (build 26.0.1+8-34)
Java HotSpot(TM) 64-Bit Server VM (build 26.0.1+8-34, mixed mode, sharing)

C:\Users\Sandra>
```

Figura 1. Verificación de la instalación de Java mediante el comando `java -version` en la terminal de Windows.

Como se observa en la figura, el sistema confirma la versión de Java instalada (26.0.1) junto con el entorno de ejecución (Java SE Runtime Environment), lo que garantiza que Android Studio va a poder compilar proyectos sin inconvenientes.

2.2 Instalación y Configuración de Android Studio

Descargué Android Studio desde el sitio oficial de desarrolladores de Android (developer.android.com/studio) y completé la instalación con el tipo de configuración Standard, que descarga automáticamente el Android SDK, las herramientas de compilación (Build Tools) y un emulador de dispositivo virtual (Android Virtual Device).

2.3 Creación del Proyecto

Dentro de Android Studio, creé un proyecto nuevo utilizando la plantilla Empty Activity, la más simple que ofrece el asistente, ideal para un primer ejercicio porque genera solamente lo indispensable: una actividad principal y una función de saludo, sin componentes adicionales que puedan confundir en esta primera aproximación.

2.4 Configuración del Proyecto

Configuré el proyecto con los siguientes parámetros:

Parámetro	Valor configurado
Nombre del proyecto	TRUEQUEMATCH
Nombre del paquete	com.example.truequematch
Lenguaje	Kotlin
Plantilla	Empty Activity
Dispositivo virtual	Pixel 7, API 36

2.5 Creación y Codificación de la Actividad Principal

La plantilla Empty Activity generó automáticamente el archivo MainActivity.kt con una actividad principal (MainActivity) que hereda de ComponentActivity, y una función Composable llamada Greeting encargada de mostrar un texto en pantalla. Configuré esta función para que reciba el mensaje a mostrar y lo despliegue usando el componente Text de Jetpack Compose, el sistema moderno de Google para construir interfaces en Android de forma declarativa.

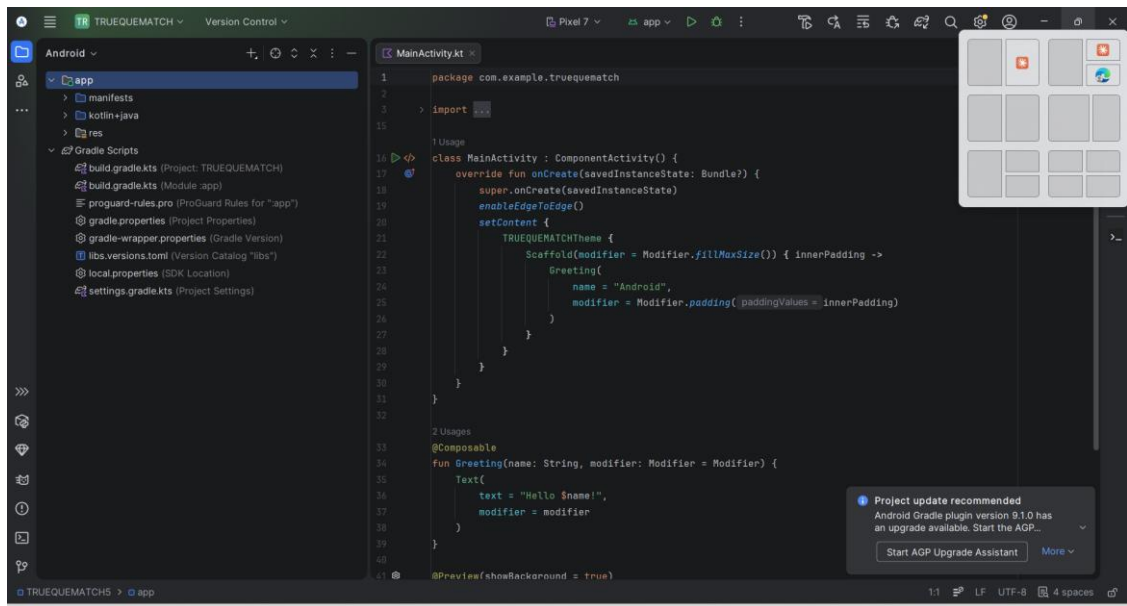


Figura 2. Proyecto TRUEQUEMATCH abierto en Android Studio, mostrando la estructura de carpetas y el código de MainActivity.kt.

Como se observa en la figura, el código define la función `Greeting(name: String)`, que arma un texto con el patrón "Hello \$name!" usando interpolación de cadenas de Kotlin. Dentro de `onCreate()`, el método `setContent {}` invoca esta función pasándole el valor "Android" como parámetro, lo que da como resultado el mensaje "Hello Android!" en pantalla.

2.6 Compilación y Ejecución de la Aplicación

Con el proyecto configurado, presioné el botón Run de Android Studio, que desencadena el proceso de compilación: Gradle transforma el código Kotlin en bytecode ejecutable, empaqueta los recursos y genera un APK de depuración que se instala automáticamente en el emulador seleccionado (Pixel 7, API 36).

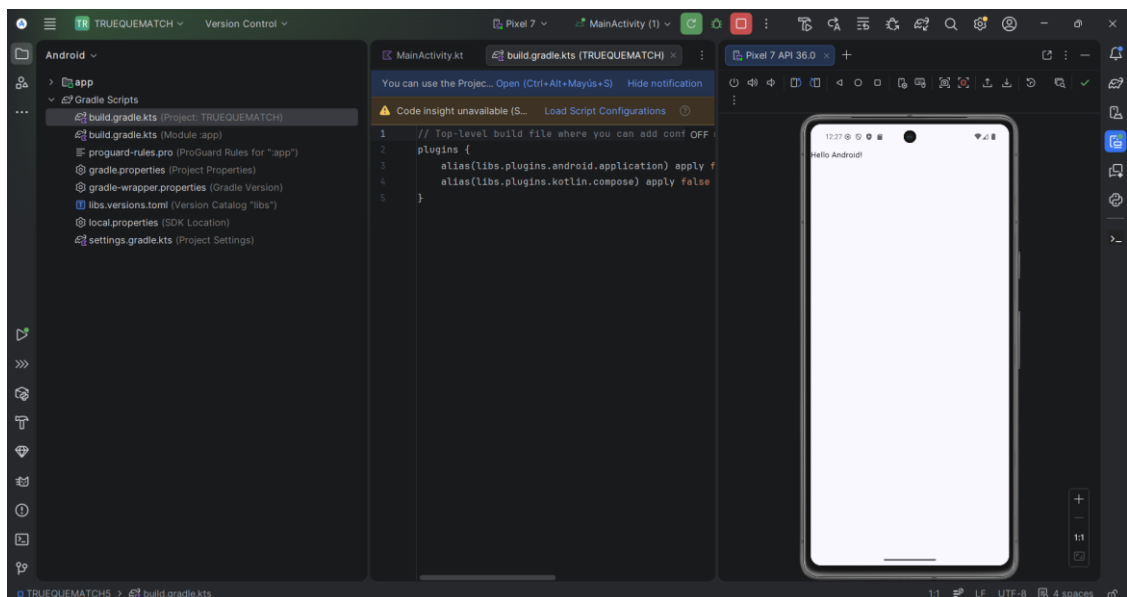


Figura 3. Aplicación TRUEQUEMATCH compilada y ejecutándose en el emulador Android Pixel 7, mostrando el mensaje "Hello Android!" en pantalla.

La figura confirma la ejecución exitosa de la aplicación: el emulador muestra la pantalla de la app con el texto "Hello Android!" renderizado correctamente en la esquina superior izquierda, sin errores de compilación, lo que demuestra que el ciclo completo (instalación de Java, instalación de Android Studio, creación del proyecto, codificación de la actividad, compilación y ejecución) se completó satisfactoriamente.

2.7 Relación con el Proyecto Trueque Match

Este ejercicio básico complementa el trabajo ya realizado en la evidencia GA5-220501095-AA1-EV08, donde la interfaz completa de Trueque Match se desarrolló usando React Native con Expo, y se vinculó igualmente con Android Studio para su visualización y depuración sobre el mismo emulador Pixel 7. Dominar la creación de un proyecto Android nativo desde cero, como el realizado en esta sección, refuerza la comprensión de lo que ocurre por debajo de frameworks como React Native, que en última instancia compilan hacia el mismo tipo de proyecto Android nativo aquí documentado.

Conclusiones

Este taller permitió consolidar, tanto en la teoría como en la práctica, los fundamentos del desarrollo de aplicaciones móviles para Android. La investigación de la Sección 1 estableció el vocabulario y los conceptos esenciales (Android, APK, SDK, lenguajes de programación, IDEs, multiusuario, mínimo privilegio y componentes de una app) necesarios para entender cómo funciona el ecosistema Android por dentro.

La Sección 2 llevó esos conceptos a la práctica: instalar Java, instalar Android Studio, crear un proyecto desde cero, codificar una actividad y ejecutarla mostrando un mensaje en pantalla, sin depender de herramientas intermedias como Expo. Esto demuestra que, más allá de usar React Native para el desarrollo principal de Trueque Match, entiendo la base nativa de Android sobre la que ese framework en realidad se apoya.

Referencias

Android Developers. (2024). Meet Android Studio.

<https://developer.android.com/studio/intro>

Google. (2025). Android Core Principles and Security Overview. Android Open Source Project.

Google. (2023). Android Developers Documentation. Google LLC.

<https://developer.android.com/docs>

Kotlin Foundation. (2024). Kotlin Documentation.

<https://kotlinlang.org/docs/home.html>

SENA. (2023). Componente Formativo: Análisis y Desarrollo de Software. Servicio Nacional de Aprendizaje.